TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

AMCS1043 Database Development and Applications
Assignment Submission Form

Programme	DCS1S2
Tutorial Group	G7
Lecturer / Tutor	Ms. Ong Phaik Ling
Submission Date	20 December 2025

No.	Student Name (Please fill in the names in ascending order)	Student ID	Signature	Contribution (%)	Marks
1.	Annie Yap Xin Yi	25WMD07613		20	
2.	Joshua A/L Albert Leo	25WMD07617		20	
3.	Looi Wei Xiang	25WMD07682		20	
4.	Soh Jia Le	25WMD07702		20	
5.	Yuki Soo Zi Qi	25WMD07343		20	
				100	

Feedback / Comments:

Appendix 2

TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

AMCS1043 Database Development and Applications

DCS1S2G7

DCS1S2G7_Beverage_Mixue_Slide

Plagiarism Statement

Read, complete, and sign this statement to be submitted with the written report.

We confirm that the submitted work is all our own work and is in our own words.

	Name (Block Capitals) (Please fill in the names)	Registration No.	Signature
1.	ANNIE YAP XIN YI	25WMD07613	
2.	JOSHUA A/L ALBERT LEO	25WMD07617	
3.	LOOI WEI XIANG	25WMD07628	
4.	SOH JIA LE	25WMD07702	
5.	YUKI SOO ZI QI	25WMD07343	

Tutorial Group : .....G7.....

Date : .....20/12/2025.....

Assignment Assessment Rubrics

Programme: DCS1S2 **Tutorial Group : 7**

Criteria	Marks	Excellent	Good	Average	Poor	Score
GROUP						
(1) Organisation background	10	Very high level organisational analysis and research, covering all relevant details. Very clear and excellent description of problem statement and solution. 9 – 10	High level organisational analysis and research, covering mostly relevant details. Good and clear description of problem statement and solution. 6 – 8	Some organisational analysis and research, barely covering relevant details. Acceptable and clear description of problem statement and solution. 3 – 5	Lack of organisational analysis and research, covering details mostly not relevant. Insufficient and unclear description of problem statement and solution. 0 – 2	
(2) Business rules	10	Excellent business rules. 9 – 10	Good business rules. 6 – 8	Acceptable business rules. 3 – 5	Poor or incorrect business rules. 0 – 2	
(3) Database Design (ERD)	10	Excellent database design. 9 – 10	Good database design. 6 – 8	Acceptable database design. 3 – 5	Poor database design. 0 – 2	
(4) Create database tables in Oracle	20	Excellent design of Oracle database with all requirements met. 17 – 20	Good design of Oracle database with major requirements met. 11 – 16	Acceptable design of Oracle database with some requirements met. 5 – 10	Poor design of Oracle database and does not meet most of the requirements shown. 0 – 4	
(5) Create records	20	Excellent and sufficient records resemble real world data. 17 – 20	Good and sufficient records resemble real world data. 11 – 16	Acceptable and some adequate records resemble real world data. 5 – 10	Poor and unrealistic data. 0 – 4	
(7) Assignment report	10	Excellent writing style. Messages and answers are clear and	Good writing style. Messages and answers sometimes	Average writing style. Messages and answers are	Writing style is poor. Answers are poorly conveyed.	

		effective. Full compliance of format. 9 – 10	lack clarity. Close compliance of format. 6 – 8	poorly conveyed. Minimum compliance of format. 3 – 5	Non-compliance of format. 0 – 2	
Total	80					
INDIVIDUAL						
(6) Create query	10	Excellent and correct design of query statement based on the purpose/ scenario with excellent formatting. 9 – 10	Good and correct design of query statement based on the purpose/ scenario with appropriate formatting. 6 – 8	Average and correct design query statement based on the purpose/ scenario with appropriate formatting 3 – 5	Poor and incorrect design query statement with poor formatting. 0 – 2	
(8) Presentation	10	Excellent communication skills. Messages and answers are clear and effective. 9 – 10	Good communication skills. Messages and answers sometimes lack clarity. 6 – 8	Average communication skill. Messages and answers are poorly conveyed. 3 – 5	Use of language is ineffective; communication is generally poor. Answers are poorly conveyed. 0 – 2	
Total	20					

Name **(Please fill in the names in ascending order)**	**Query (10%)**	**Presentation (10%)**	**Marks (Task 6 & 8)**	**Remark (if any)**
1.Annie Yap Xin Yi				
2.Joshua A/L Albert Leo				
3.Looi Wei Xiang				
4.Soh Jia Le				
5.Yuki Soo Zi Qi				
6.				

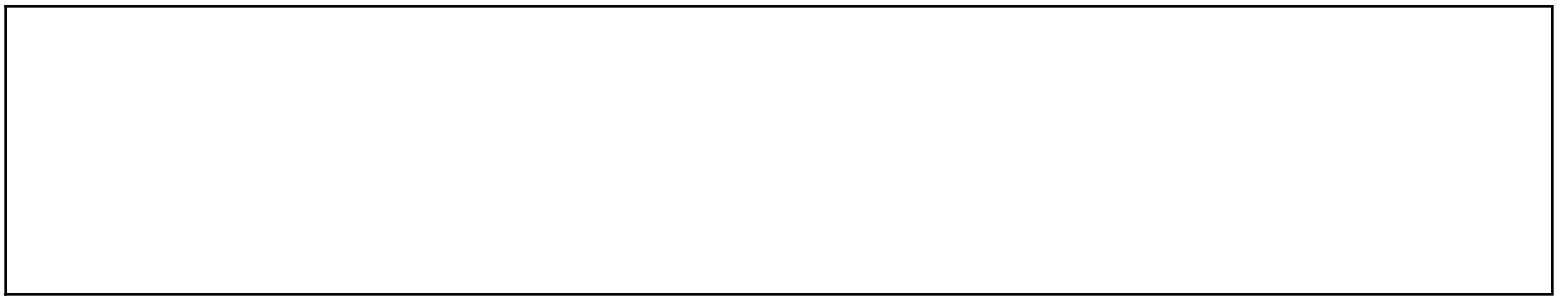


TABLE OF CONTENTS

1.0 Organisation Background....9
1.1 Brief overview of the organisation's background.... 9
1.2 Opportunities faced by the organisation.... 9
1.3 Opportunity statement.... 9
1.4 Description of the opportunity.... 8
1.5 Aims of the proposed database solution....9
1.6 Objectives of the proposed database solution....9
2.0 Develop business rules....10
2.1 Each order must belong to exactly one store location.... 10
2.2 Each order must be processed by exactly one staff member.... 10
2.3 Menu items must belong to only one category.... 10
2.4 An order must contain at least one order item.... 10
2.5 Prices of menu items are fixed by the store and cannot be altered at the time of order..10
3.0 Database design.... 11
3.1 Entity-Relationship Diagram (ERD)....11
3.2 Database Design Language (DBDL):....11
4.0 Create database tables in Oracle.... 12
4.1 store_location.... 12
4.2 staff.... 12
4.3 customer.... 13
4.4 orders....14
4.5 category.... 14
4.6 menu....14
4.7 orders_detail.... 15
5.0 Create records.... 16
5.1 store_location.... 16
5.2 staff.... 16
5.3 customer.... 17
5.4 orders....18
5.5 category.... 18
5.6 menu....19
5.7 order_detail....19
6.0 Create Query.... 20
6.1 Staff Order Performance Report (Annie Yap Xin Yi)....20
6.1.1 Query Purpose....20
6.1.2 Query....20
6.1.3 Test Case....22
6.1.3.1 Result....22
6.1.3.2 Result....22

6.2 Store Sales Report in 2024 between 2 dates(Joshua A/L Albert Leo)..... 23
6.2.1 Purpose.....23
6.2.2 Query..... 23
6.2.2 Test Case.....24
6.2.3.1 Result.....24
6.3 Monthly Top Selling Menu Item Report (Looi Wei Xiang)..... 25
6.3.1 Query purpose..... 25
6.3.2 Query..... 25
6.4 Most Ordered Category Report (Soh Jia Le)..... 27
6.4.1 Query Purpose.....27
6.4.2 Query..... 27
6.4.3 Test Case.....29
6.4.3.1 Result.....29
6.4.3.2 Result.....29
6.5 Daily Sales Report (Yuki Soo Zi Qi).....30
6.5.1 Query Purpose.....30
6.5.2 Query..... 30
6.5.3 Test Case.....32
6.5.3.1 Result.....32

1.0 Organisation Background

1.1 Brief overview of the organisation’s background

Mixue is a fast-growing beverage and dessert franchise established in 1997 in China. It offers affordable bubble tea, fruit tea, smoothies, and ice cream products. The organisation operates through physical retail outlets, where staff prepare drinks on demand and serve walk-in customers. Its business operations include managing customer orders, processing payments, and ensuring consistent product quality across different store branches. Mixue provides a variety of products such as milk tea, fruit tea, smoothies, and its signature ice cream to customers through offline, in-store purchases.

1.2 Opportunities faced by the organisation

As Mixue expands into multiple store locations, the organisation has the opportunity to employ a database system to manage and improve the operations more efficiently. For example, database systems allow for centralized data storage, which can handle and accommodate the growing data volume of information from each Mixue store location that include customer, staff, menu and other relevant information. In addition, a database system can retrieve and analyze data across multiple store locations, so it can help to consolidate information and obtain a comprehensive view of Mixue operation from each store location.

1.3 Opportunity statement

Mixue has the opportunity to enhance its walk-in ordering operations by developing a database system that manages information on the store locations, customer’s orders, items from the menu, and staff details. This allows the organisation to streamline data management and support better business analysis. Moreover, it also provides insight into the organization's operations.

1.4 Description of the opportunity

By introducing a database that includes a Store_Location entity, Mixue can systematically track orders made in each outlet and better understand the performance of individual branches. The system will organise essential data such as order details, total sales, and menu item popularity, all linked to specific store locations. This opportunity supports more accurate reporting, simplifies branch-level management, and enables the company to evaluate which store locations perform best based on sales trends and customer demand.

1.5 Aims of the proposed database solution

The proposed database aims to support Mixue in managing walk-in order information across multiple store locations. It intends to ensure that important data such as orders, staff, menu items, and store branches is stored accurately and consistently, at the same time enhancing daily operational efficiency and decision-making.

1.6 Objectives of the proposed database solution

The objectives of this database system are to:

- Store structured information about each store location, including name and address.
- Record all offline customer orders accurately and associate each order with its respective outlet.
- Manage menu details, categories, and pricing in a standardised format across all branches.
- Track staff involvement in each order for accountability and performance analysis.
- Produce reports such as top-selling items per location, total sales per location, and overall outlet performance comparisons.

2.0 Develop business rules

2.1 Each order must belong to exactly one store location

All offline orders must be recorded under the specific Mixue outlet where the transaction occurs. A store location cannot be deleted if existing orders are linked to it.
Assumption: Each Mixue outlet operates independently but uses the same central system.

2.2 Each order must be processed by exactly one staff member

Each staff member must serve at least one order, each order is handled by one staff member. An order cannot be shared among multiple staff members.
Constraint: staff_id in **Orders** table must not be NULL.

2.3 Menu items must belong to only one category

Every menu item must be assigned to exactly one product category (e.g., Milk Tea, Ice Cream, Fruit Tea, Smoothies). Categories cannot be removed if menu items still depend on them.
Constraint: category_id in **Menu** must be valid and unique per item.

2.4 An order must contain at least one order item

Each order must have at least one or more items, and the quantity for each item must be more than zero.
Constraint: No empty orders are allowed.

2.5 Prices of menu items are fixed by the store and cannot be altered at the time of order

The price stored in the Menu table must be used during ordering. Staff are not allowed to modify item prices during order creation.
Constraint: The price recorded in Order_Item must be retrieved from the Menu automatically.

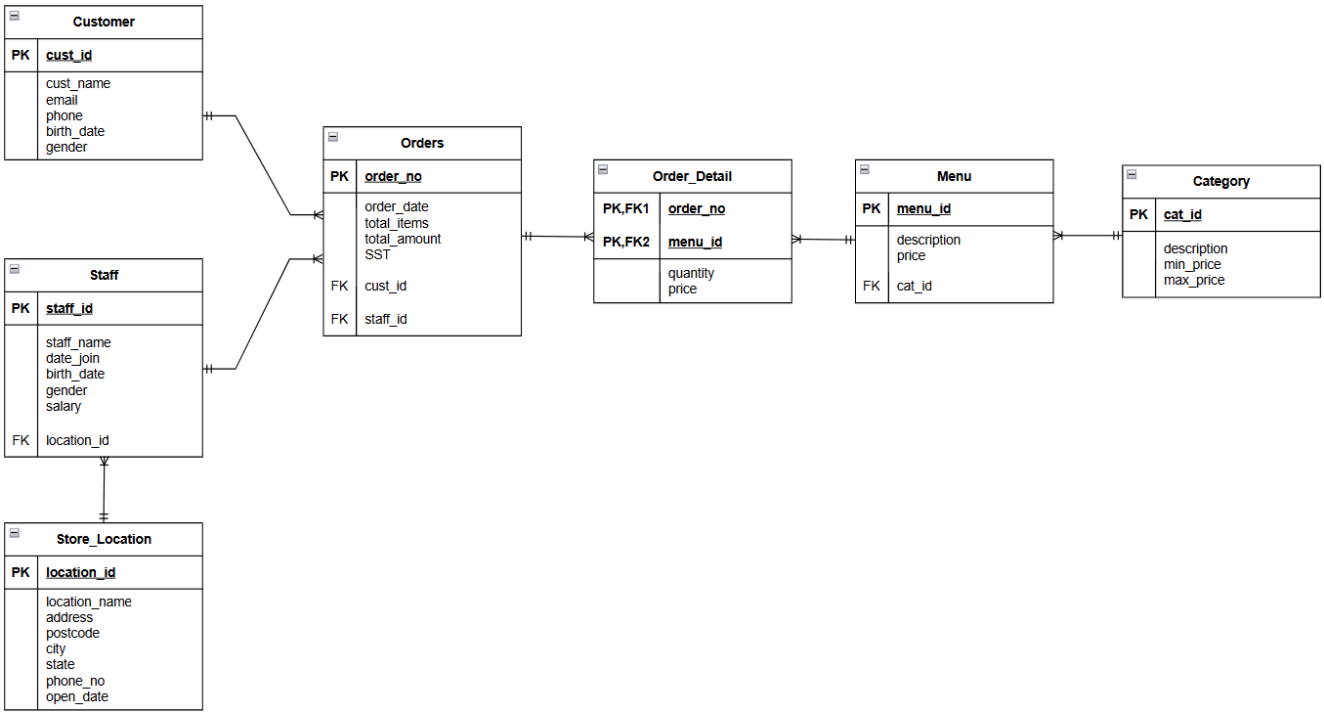
2.6 Each store location must have at least one staff

Each store location has at least one staff member, each staff member is assigned to one store location.

Constraint: The location_id of staff cannot be null and it has to reference a location_id from the Store_Location table

3.0 Database design

3.1 Entity-Relationship Diagram (ERD)



3.2 Database Design Language (DBDL):

Customer(cust_id, cust_name, email, phone, birth_date, gender)
Staff (staff-id, staff_name, salary, birth_date, date_join, gender, location_id*)
Store_Location (location_id, location_name, address, postcode, city, state, phone_no, open_date)
Orders(order_no, order_date, cust_id*, staff_id*, total_items, total_amount, SST)
Order_Detail (order_no*, menu_id*, quantity, price)
Menu(menu_id, description, price, cat_id*)
Category(cat_id, description, min_price, max_price)

4.0 Create database tables in Oracle

4.1 store_location

```
create table store_location (
   location_id   varchar(4)  not null,
   location_name varchar(35),
   address varchar(80),
   postcode varchar(5),
   city varchar(15),
   state varchar(15),
   phone_no varchar(12),
   open_date date,
primary key(location_id)
);
```

```
SQL> create table store_location (
  2     location_id     varchar(4)  not null,
  3     location_name varchar(35),
  4     address varchar(80),
  5     postcode varchar(5),
  6     city varchar(15),
  7     state varchar(15),
  8     phone_no varchar(12),
  9     open_date date,
 10  primary key(location_id)
 11  );

Table created.

SQL>
```

Diagram 4.1 shows the created table *store_location*.

4.2 staff

```
create table staff (
  staff_id        varchar(4)   not null,
  staff_name   varchar(25) not null,
  salary       number(10,2),
  birth_date   date,
  date_join    date,
  gender       varchar(1),
  location_id  varchar(4),
primary key(staff_id),
foreign key(location_id) references store_location(location_id)
);
```

```
SQL> create table staff (
  2     staff_id        varchar(4)   not null,
  3     staff_name   varchar(25) not null,
  4     salary       number(10,2),
  5     birth_date   date,
  6     date_join    date,
  7     gender       varchar(1),
  8     location_id  varchar(4),
  9  primary key(staff_id),
 10  foreign key(location_id) references store_location(location_id)
 11  );

Table created.

SQL>
```

Diagram 4.2 shows the created table *staff*.

4.3 customer

```
create table customer (
   cust_id        varchar(4)   not null,
   cust_name      varchar(25) not null,
   email          varchar(35),
   phone          varchar(14),
   birth_date     date,
   gender         char(1),
primary key(cust_id),
constraint chk_email check (REGEXP_LIKE(email,'^[a-zA-Z]\w+@(\S+)$')),
constraint chk_gender check (UPPER(gender) in ('M','F'))
);
```

```
SQL> create table customer (
  2     cust_id varchar(4)   not null,
  3     cust_name       varchar(25) not null,
  4     email   varchar(35),
  5     phone   varchar(14),
  6     birth_date   date,
  7     gender  char(1),
  8  primary key(cust_id),
  9  constraint chk_email check (REGEXP_LIKE(email,'^[a-zA-Z]\w+@(\S+)$')),
 10  constraint chk_gender check (UPPER(gender) in ('M','F'))
 11  );

Table created.

SQL>
```

Diagram 4.3 shows the created table *customer*.

4.4 orders

```
create table orders (
  order_no          number(7) not null,
  order_date        date,
  cust_id           varchar(4),
  staff_id          varchar(4),
  total_items       number(2),
  total_amount              number(7,2),
  SST               number(6,2),
primary key(order_no),
foreign key(cust_id)  references customer(cust_id),
foreign key(staff_id) references staff(staff_id)
);
```

```
SQL> create table orders (
  2     order_no                number(7) not null,
  3     order_date              date,
  4     cust_id         varchar(4),
  5     staff_id                varchar(4),
  6     total_items             number(2),
  7     total_amount            number(7,2),
  8     SST                     number(6,2),
  9  primary key(order_no),
 10  foreign key(cust_id)  references customer(cust_id),
 11  foreign key(staff_id) references staff(staff_id)
 12  );

Table created.

SQL>
```

Diagram 4.4 shows the created table *orders*.

4.5 category

```
create table category (
  cat_id        varchar(4)  not null,
  description   varchar(30) not null,
  min_price     number(6,2),
  max_price     number(6,2),
primary key(cat_id)
);
```

```
SQL> create table category (
  2     cat_id  varchar(4)  not null,
  3     description     varchar(30) not null,
  4     min_price       number(6,2),
  5     max_price       number(6,2),
  6  primary key(cat_id)
  7  );

Table created.

SQL>
```

Diagram 4.5 shows the created table *category*.

4.6 menu

```
create table menu (
  menu_id               varchar(4)  not null,
  description           varchar(40) not null,
  price         number(6,2),
  cat_id                varchar(5),
primary key(menu_id),
foreign key(cat_id) references category(cat_id)
);
```

```
SQL> create table menu (
  2     menu_id         varchar(4)  not null,
  3     description             varchar(40) not null,
  4     price           number(6,2),
  5     cat_id          varchar(5),
  6  primary key(menu_id),
  7  foreign key(cat_id) references category(cat_id)
  8  );

Table created.

SQL>
```

Diagram 4.6 shows the created table *menu*.

4.7 orders_detail

```
create table order_detail (
   order_no      number(7)  not null,
   menu_id       varchar(5) not null,
   quantity      number(2),
   price number(6,2),
primary key(order_no,menu_id),
foreign key(order_no) references orders(order_no),
foreign key(menu_id) references menu(menu_id)
);
```

```
SQL> create table order_detail (
  2     order_no        number(7)  not null,
  3     menu_id varchar(5) not null,
  4     quantity        number(2),
  5     price   number(6,2),
  6  primary key(order_no,menu_id),
  7  foreign key(order_no) references orders(order_no),
  8  foreign key(menu_id) references menu(menu_id)
  9  );

Table created.

SQL>
```

Diagram 4.7 shows the created table *order_detail*.

5.0 Create records

5.1 store_location

INSERT INTO store_location VALUES ('SH04','Mixue Puchong Utama','35, Jalan Bpu 2, Bandar Puchong Utama','47100','Puchong','Selangor','018-4646008',DATE '2024-01-07');

INSERT INTO store_location VALUES ('HL67','Mixue Taiping','76D, Jalan Tupai','34000','Taiping','Perak ','018-9867188',DATE '2024-11-26');

INSERT INTO store_location VALUES ('AK47','Mixue Plaza Ushawan Danau Kota','No 31 01, Plaza Usahawan Crystal Ville, Off, Jalan Danau Niaga 1, Danau Kota','53300','Kuala Lumpur','WP Kuala
Lumpur','011-56515760',DATE '2023-12-01');

INSERT INTO store_location VALUES ('JB01','Mixue JB City Square','Lot 3-12, JB City Square','80200','Johor Bahru','Johor','07-2213456',DATE '2023-05-15');

INSERT INTO store_location VALUES ('PG02','Mixue Queensbay','Lot 2F-45, Queensbay Mall','11900','Bayan Lepas','Penang','04-6123456',DATE '2023-08-20');

Total records: 5

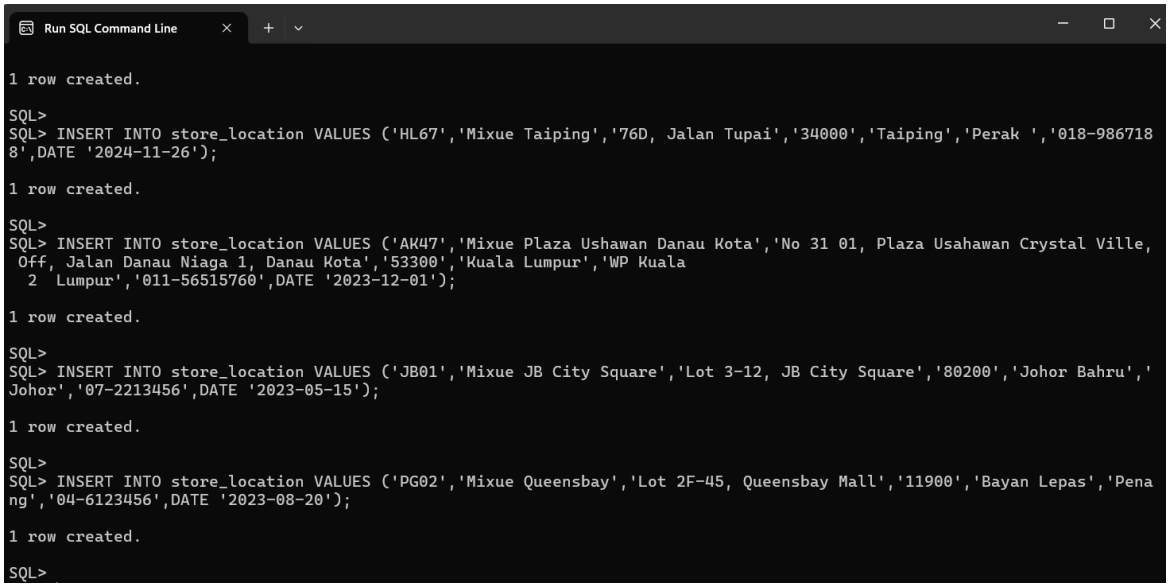


Diagram 5.1 shows the inserted values in the *store_location* table.

5.2 staff

INSERT INTO staff VALUES ('S101','Joshua',10000,DATE '2007-03-25',DATE '2024-06-01','M','SH04');

INSERT INTO staff VALUES ('S102','Yuki',8000,DATE '2007-06-26',DATE '2024-06-03','F','SH04');

INSERT INTO staff VALUES ('S103','Annie',8000,DATE '2006-05-02',DATE '2024-06-04','F','AK47');

INSERT INTO staff VALUES ('S104','Jia Le',8000,DATE '2007-06-27',DATE '2024-06-04','M','AK47');

INSERT INTO staff VALUES ('S105','Wei Xiang',8000,DATE '2006-04-22',DATE '2024-06-06','M','HL67');

INSERT INTO staff VALUES ('S106','Aiman',7500,DATE '2006-03-15',DATE '2024-06-10','M','HL67');

INSERT INTO staff VALUES ('S107','Siti',7500,DATE '2007-07-18',DATE '2024-06-10','F','HL67');

INSERT INTO staff VALUES ('S108','Daniel',8200,DATE '2005-01-11',DATE '2024-06-12','M','JB01');

INSERT INTO staff VALUES ('S109','Nabila',8200,DATE '2006-09-03',DATE '2024-06-12','F','JB01');

INSERT INTO staff VALUES ('S110','Kevin',8000,DATE '2005-11-09',DATE '2024-06-15','M','PG02');

Total records: 10

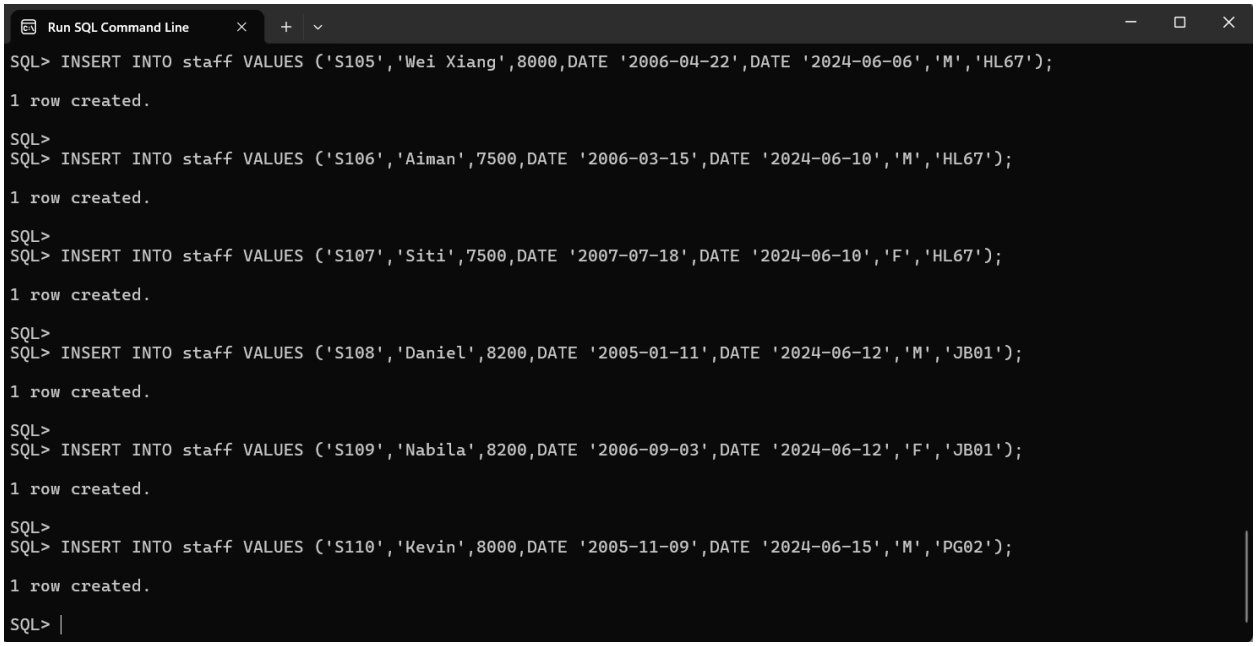


Diagram 5.2 shows the inserted values in the *staff* table.

5.3 customer

INSERT INTO customer VALUES ('C001','John','john1990@gmail.com','0122617345',DATE '1990-05-25','M');

INSERT INTO customer VALUES ('C002','Craig','craig1970@gmail.com','0102213990',DATE '1970-03-04','M');

INSERT INTO customer VALUES ('C003','Charles','charles2006@gmail.com','0102213990',DATE '2006-06-09','M');

INSERT INTO customer VALUES ('C004','Joseph','joseph2008@gmail.com','0102245869',DATE '2008-06-14','M');

INSERT INTO customer VALUES ('C005','Alex','alex2006@gmail.com','0125348795',DATE '2006-12-16','F');

INSERT INTO customer VALUES ('C006','Elena','alena1999@gmail.com','0123356821',DATE '1999-11-27','F');

INSERT INTO customer VALUES ('C007','Olivia','olivia1997@gmail.com','0123479231',DATE '1997-05-21','F');

INSERT INTO customer VALUES
('C008','Christina','christina2001@gmail.com','0103950248',DATE '2001-02-07','F');

INSERT INTO customer VALUES ('C009','Grace','grace2009@gmail.com','0103750629',DATE '2009-10-05','F');

INSERT INTO customer VALUES ('C010','Ethan','ethan2000@gmail.com','0128899001',DATE '2000-10-10','M');

Total records: 10



Diagram 5.3 shows the inserted values in the *customer* table.

5.4 orders

INSERT INTO orders VALUES (1000,'12-JAN-2024','C001','S110',2,12.5,.75);
INSERT INTO orders VALUES (1001,'07-JAN-2024','C001','S101',2,11,.66);
INSERT INTO orders VALUES (1002,'13-JAN-2024','C002','S108',2,11.5,.69);
INSERT INTO orders VALUES (1003,'24-JAN-2024','C002','S107',2,10.5,.63);
INSERT INTO orders VALUES (1004,'27-JAN-2024','C003','S104',2,9,.54);
INSERT INTO orders VALUES (1005,'26-JAN-2024','C003','S108',2,10,.6);
INSERT INTO orders VALUES (1006,'14-JAN-2024','C004','S109',2,13,.78);
INSERT INTO orders VALUES (1007,'22-JAN-2024','C004','S110',2,10,.6);

INSERT INTO orders VALUES (1008,'21-JAN-2024','C005','S110',2,10.5,.63);
INSERT INTO orders VALUES (1009,'25-JAN-2024','C005','S107',2,8.5,.51);
Total records: 240

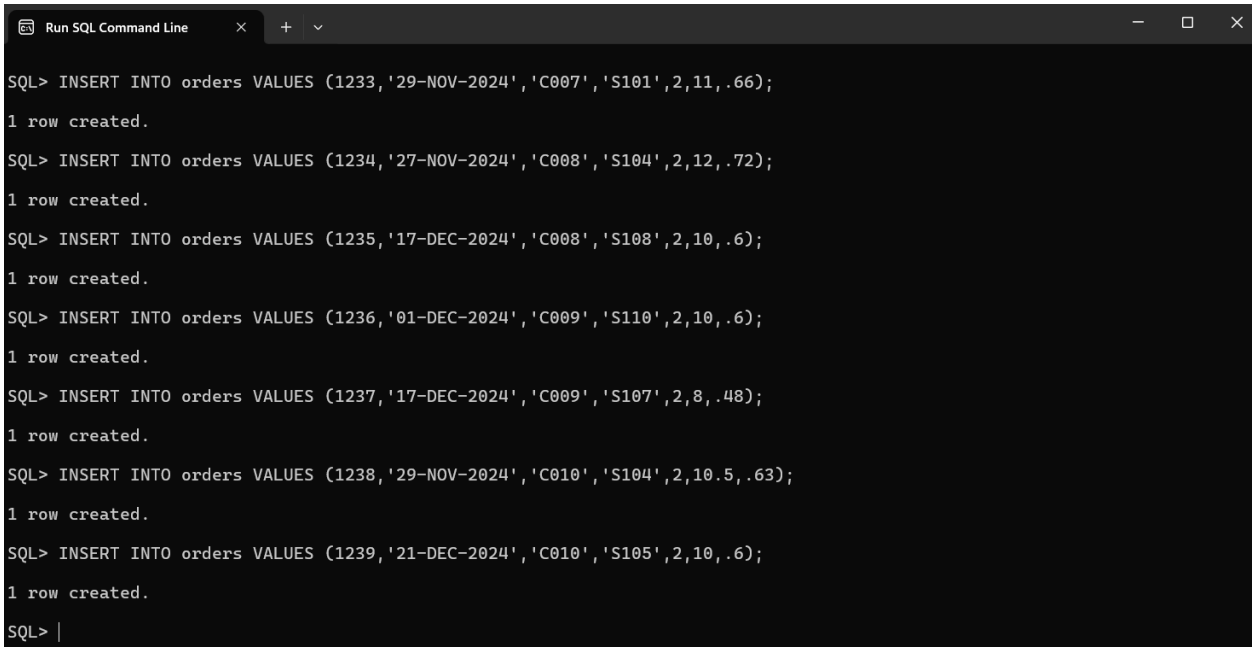


Diagram 5.4 shows the inserted values in the *orders* table.

5.5 category

INSERT INTO category VALUES ('IC01','Ice Cream Series',2,5);
INSERT INTO category VALUES ('FT01','Fruit Tea Series',3,7);
INSERT INTO category VALUES ('MT01','Milk Tea Series',4.5,7);
INSERT INTO category VALUES ('CF01','Coffee Series',5.5,5.5);
Total records: 4

```
SQL> INSERT INTO category VALUES ('IC01','Ice Cream Series',2,5);

1 row created.

SQL> INSERT INTO category VALUES ('FT01','Fruit Tea Series',3,7);

1 row created.

SQL> INSERT INTO category VALUES ('MT01','Milk Tea Series',4.5,7);

1 row created.

SQL> INSERT INTO category VALUES ('CF01','Coffee Series',5.5,5.5);

1 row created.

SQL>
```

Diagram 5.5 shows the inserted values in the *category* table.

5.6 menu

INSERT INTO menu VALUES ('A1','MIXUE Ice Cream',2,'IC01');
INSERT INTO menu VALUES ('A2','Strawberry Mi-Shake',5,'IC01');
INSERT INTO menu VALUES ('A3','Peach Mi-Shake',5,'IC01');
INSERT INTO menu VALUES ('A4','Boba Mi-Shake',5,'IC01');
INSERT INTO menu VALUES ('A5','Super Boba Sundae',5,'IC01');
INSERT INTO menu VALUES ('A6','Super Mango Sundae',5,'IC01');
INSERT INTO menu VALUES ('A7','Strawberry Crispy Sundae',5,'IC01');
INSERT INTO menu VALUES ('A8','O-Crispy Sundae',5,'IC01');
INSERT INTO menu VALUES ('B1','Fresh Lemonade',3,'FT01');
INSERT INTO menu VALUES ('B2','Kiwi Jasmine Tea',5,'FT01');
Total records: 27

Mixue Malaysia Menu: http://mixuemalaysia.com/menu

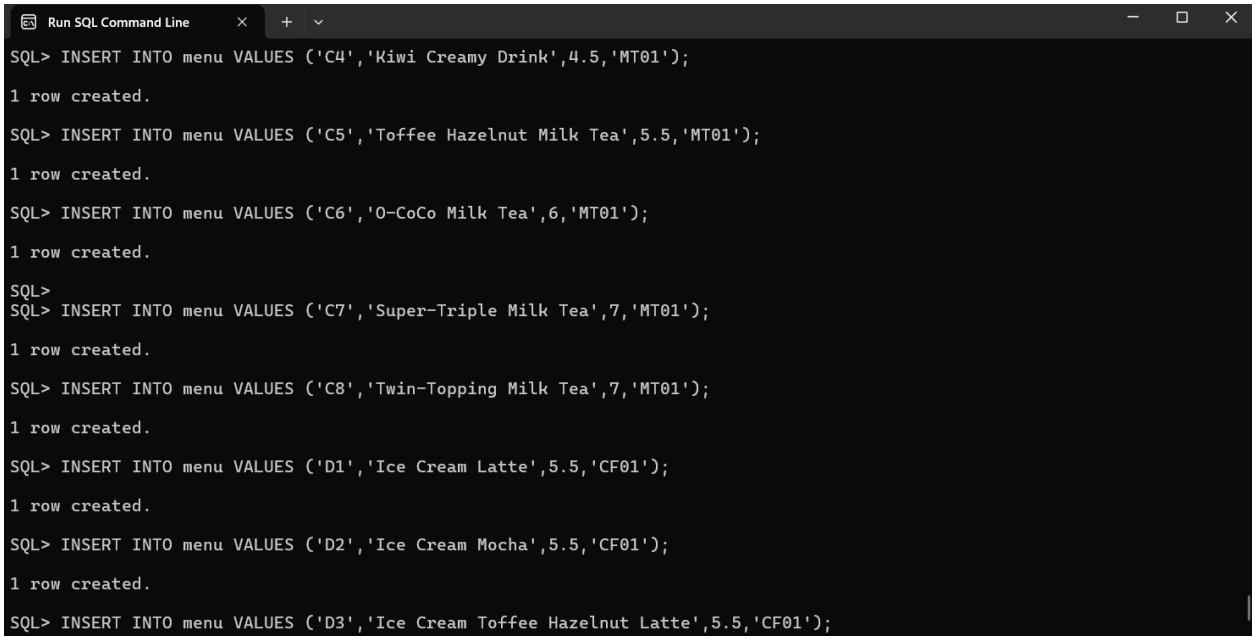


Diagram 5.6 shows the inserted values in the *menu* table.

5.7 order_detail

INSERT INTO order_detail VALUES (1000,'C5',1,5.5);
INSERT INTO order_detail VALUES (1000,'C7',1,7);
INSERT INTO order_detail VALUES (1001,'A6',1,5);
INSERT INTO order_detail VALUES (1001,'C6',1,6);
INSERT INTO order_detail VALUES (1002,'B5',1,6);
INSERT INTO order_detail VALUES (1002,'D1',1,5.5);
INSERT INTO order_detail VALUES (1003,'A4',1,5);
INSERT INTO order_detail VALUES (1003,'D3',1,5.5);
INSERT INTO order_detail VALUES (1004,'B1',1,3);
INSERT INTO order_detail VALUES (1004,'B8',1,6);
Total records: 480

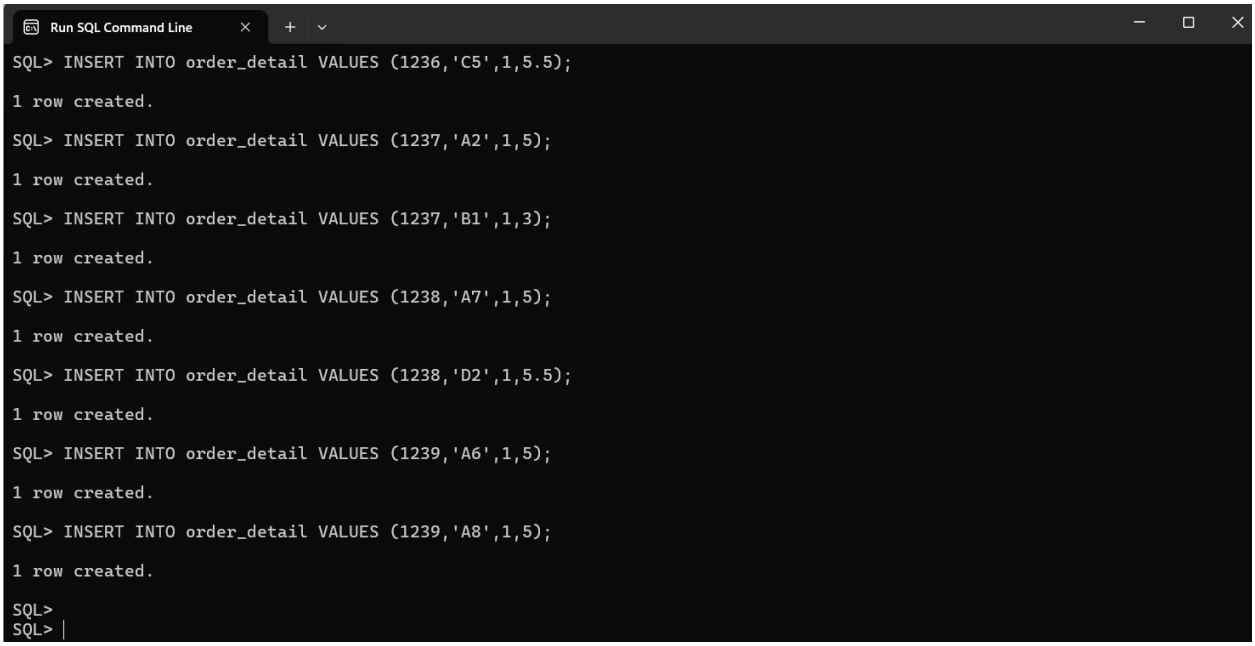


Diagram 5.7 shows the inserted values in the *order-detail* table.

6.0 Create Query

6.1 Staff Order Performance Report (Annie Yap Xin Yi)

6.1.1 Query Purpose

This query generates a staff order performance report showing who handled the most customer orders. It connects data from three tables: staff information, order records, and store location details.

The script first asks users to enter filter options for the month and the store location. Based on these inputs, the query calculates each staff member's total orders and sales amount, then ranks them from highest to lowest order count. The results are displayed in a formatted table with proper headings and a total summary at the bottom.

This report helps managers identify top-performing employees, make fair evaluations, and improve store operations based on actual order processing data.

6.1.2 Query

```
-- Clear previous settings
CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
TTITLE OFF
BTITLE OFF

-- Set environment
SET ECHO OFF
SET VERIFY OFF
SET FEEDBACK OFF
SET PAGESIZE 40
SET LINESIZE 90

--INPUT
PROMPT Filter Options for Staff Performance Report:
PROMPT
ACCEPT p_month CHAR PROMPT 'Enter month (1-12, or ALL): '
ACCEPT p_location CHAR PROMPT 'Enter location code (e.g.,
SH04/HL67/AK47/JB01/PG02/ALL): '
```

```
-- Report title
TTITLE CENTER 'STAFF ORDER PERFORMANCE REPORT' SKIP 1 -
      CENTER 'Top Order Handlers Analysis' SKIP 1 -
      LEFT 'Month: ' '&p_month' ' | Location: ' '&p_location' SKIP 2

-- Column formatting
COLUMN rank FORMAT 99 HEADING 'Rank '
COLUMN staff_id FORMAT A8 HEADING 'Staff ID'
COLUMN staff_name FORMAT A15 HEADING 'Staff Name'
COLUMN store FORMAT A40 HEADING 'Store'
COLUMN orders FORMAT 999 HEADING 'Orders'
COLUMN sales FORMAT '99,999.99' HEADING 'Sales(RM)'

-- Break and compute
BREAK ON REPORT
COMPUTE SUM LABEL 'TOTAL' OF orders sales ON REPORT

-- Multi-table query
SELECT
   ROW_NUMBER() OVER (ORDER BY COUNT(o.order_no) DESC) AS rank,
   s.staff_id,
   s.staff_name,
   sl.location_name AS store,
   COUNT(o.order_no) AS orders,
   SUM(o.total_amount) AS sales
FROM
   staff s
JOIN orders o
ON s.staff_id = o.staff_id
JOIN store_location sl
ON s.location_id = sl.location_id
WHERE
   (UPPER('&p_month') = 'ALL' OR EXTRACT(MONTH FROM o.order_date) =
TO_NUMBER('&p_month'))
   AND (UPPER('&p_location') = 'ALL' OR s.location_id = UPPER('&p_location'))
GROUP BY
   s.staff_id, s.staff_name, sl.location_name
ORDER BY
   orders DESC;

-- Clean up
CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
```

TTITLE OFF
BTITLE OFF

6.1.3 Test Case

Cases	Input/ Test Data	Expected Output	Test Result Status
6.1.3.1	Enter month: ALL Enter location code: ALL	Shows all staff from all stores ranked by order count for the whole year	Success
6.1.3.2	Enter month: 1 Enter location code: AK47	Shows only staff from AK47 store ranked by January orders	Success

6.1.3.1 Result

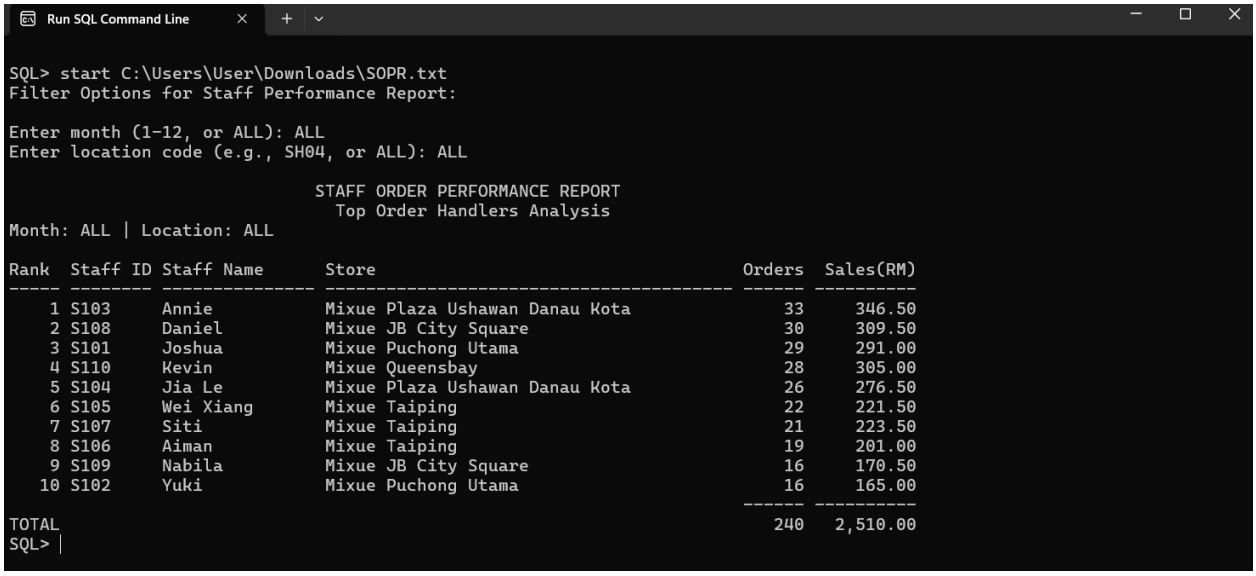


Diagram 6.1.3.1 shows all staff from all stores ranked by order count for the whole year

6.1.3.2 Result

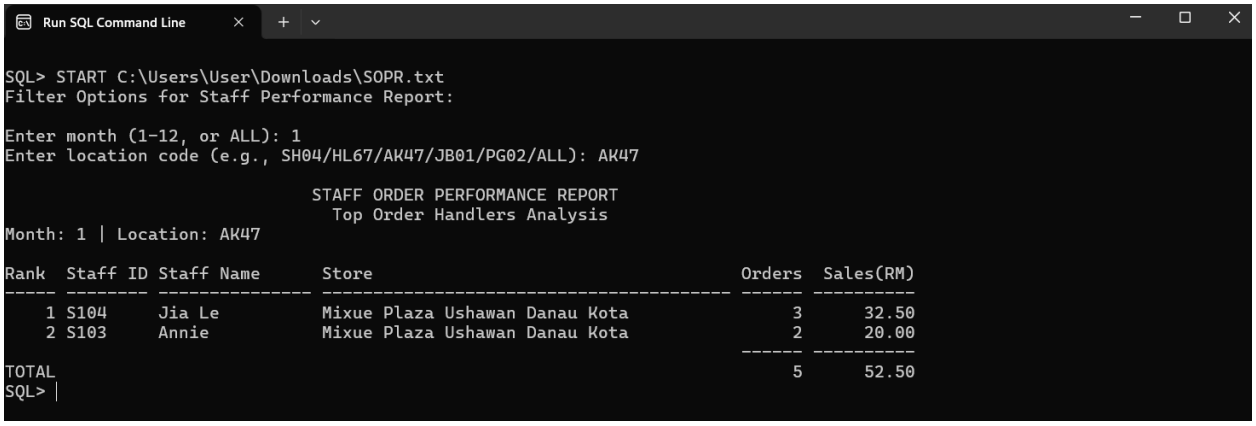


Diagram 6.1.3.2 shows only staff from AK47 store ranked by January orders

6.2 Store Sales Report in 2024 between 2 dates(Joshua A/L Albert Leo)

6.2.1 Purpose

This query shows total sales from all the stores between two dates. It connects data from 3 tables which are location of stores , staff information and order list. It uses data from 2 tables which are store location and order list.

So how does this query work? Firstly, it prompts the user to enter the starting date. Then it prompts the user to enter the ending date. Then the query filters the table to show results between the starting and ending date that the user inputted. The result displayed in a formatted table with a proper heading and computation that calculates the total sales per store.

This report helps to monitor and track store performance between 2 dates, making future decision or forecasting future revenue and support financially strategic planning

6.2.2 Query

```
--set environment--
SET VERIFY OFF
SET PAGESIZE 50000
```

```
SET LINESIZE 80

ALTER SESSION SET NLS_DATE_FORMAT = 'DD/MM/YYYY';
CL SCR

--prompt user to input--
PROMPT Store Sales Report of the Year 2024
PROMPT
ACCEPT v_start FORMAT A5 PROMPT 'Enter the start date  (DD/MM, without year): '
ACCEPT v_end   FORMAT A5 PROMPT 'Enter the end date (DD/MM, without year): '
--column formatting--
COLUMN location_id          FORMAT A11          HEADING 'Store ID'
COLUMN location_name        FORMAT A40          HEADING 'Store Name'
COLUMN total_amount         FORMAT 999999.99    HEADING 'Total Sales(RM)'
COLUMN order_date                               HEADING 'Order Date'

--set top title--
TTITLE CENTER 'Store Sales Report from &v_start to &v_end'

--break and compute--
BREAK ON location_id ON location_name
COMPUTE SUM LABEL 'Total Sales' OF total_amount ON location_id skip 2

--query--
SELECT l.location_id, l.location_name, o.order_date, o.total_amount
FROM orders o
JOIN staff s
  ON s.staff_id = o.staff_id
JOIN store_location l
  ON l.location_id = s.location_id
WHERE o.order_date BETWEEN TO_DATE('&v_start'||'/2024','DD/MM/YYYY') AND
TO_DATE('&v_end'||'/2024','DD/MM/YYYY')
ORDER BY l.location_id, o.order_date
;
--clean up--
TTITLE OFF
CLEAR BREAK
CLEAR COMPUTE
```

6.2.2 Test Case

Cases	Input/ Test Data	Expected Output	Test Result Status

6.2.3.1	Enter the start date (DD/MM): 25/7 Enter the end date (DD/MM): 27/7	Shows total items and total sales amount between 25/7/2024 and 27/7/2024	Success

6.2.3.1 Result

```
SQL> start "C:\Store Sales.txt"
```

```
Store Sales Report of the Year 2024

Enter the start date  (DD/MM, without year): 25/7
Enter the end date (DD/MM, without year): 27/7


     Store Sales Report from 25/7 to 27/7
Store ID    Store Name                              Order Date Total Sales(RM)
----------- --------------------------------------- ---------- ---------------
HL67        Mixue Simpang Taiping                   26/07/2024            8.00
*********** *************************************              ---------------
Total Sales                                                             8.00
SH04        Mixue Puchong Utama                     27/07/2024           10.50
                                                    27/07/2024            9.50
*********** *************************************              ---------------
Total Sales                                                            20.00
SQL>
```

Diagram 6.2.3.1(i) shows the sales report of each store in 2024 between 2 specific dates that enter by the users

6.3 Monthly Top Selling Menu Item Report (Looi Wei Xiang)

6.3.1 Query purpose

This query is used to find out which menu items sell the best in a selected month. It collects and combines data from the orders, order details, menu, and category tables to calculate how many times each menu item was sold.

The user can choose a month and a specific category or view all categories. The query then ranks the menu items based on total quantity sold and presents the results in a clear, well-formatted report.

This report makes it easier to understand customer preferences and helps the business plan menus, manage stock, and improve overall sales performance.

6.3.2 Query

```
SET VERIFY OFF

-- User input
ACCEPT p_month CHAR PROMPT 'Enter month (1-12, or ALL): '
ACCEPT p_category CHAR PROMPT 'Enter category code (or ALL): '

SET PAGESIZE 50000
SET LINESIZE 30000
-- Report title
TTITLE CENTER 'MONTHLY TOP SELLING MENU ITEM REPORT' SKIP 1 -
      CENTER 'Top Menu Items Analysis' SKIP 1 -
      LEFT 'Month: ' '&p_month' ' | Category: ' '&p_category' SKIP 2

-- Column formatting
COLUMN rank        FORMAT 99999        HEADING 'Rank'
COLUMN menu_id     FORMAT A8        HEADING 'Menu ID'
COLUMN menu_name   FORMAT A40       HEADING 'Menu Item'
COLUMN category    FORMAT A20       HEADING 'Category'
COLUMN quantity    FORMAT 9999      HEADING 'Total Sold'

-- Break and compute
BREAK ON REPORT
COMPUTE SUM LABEL 'TOTAL' OF quantity ON REPORT

-- Multi-table query
```

```
SELECT
    ROW_NUMBER() OVER (ORDER BY SUM(od.quantity) DESC) AS rank,
    m.menu_id,
    m.description AS menu_name,
    SUM(od.quantity) AS quantity
FROM
    orders o
JOIN order_detail od
  ON o.order_no = od.order_no
JOIN menu m
  ON od.menu_id = m.menu_id
JOIN category c
  ON m.cat_id = c.cat_id
WHERE
    (UPPER('&p_month') = 'ALL'
     OR EXTRACT(MONTH FROM o.order_date) = TO_NUMBER('&p_month'))
    AND (UPPER('&p_category') = 'ALL'
     OR c.cat_id = UPPER('&p_category'))
GROUP BY
    m.menu_id,
    m.description
ORDER BY
    rank;

-- Clean up
TTITLE OFF
CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
```

6.3.3 Test Case

Cases	Input/ Test Data	Expected Output	Test Result Status
6.3.3.1	Enter month: ALL Enter category: ALL	Shows all menu items for all months and categories, ranked by total quantity sold.	Success
6.3.3.2	Enter month: 7 Enter category: IC01	Shows all menu items for month 7 under category IC01, ranked by total quantity sold.	Success

6.3.3.1 Result

```
SQL> start "C:\InsertData\top selling.txt"
```

```
Enter month (1-12, or ALL): all
Enter category code (or ALL): all


     MONTHLY TOP SELLING MENU ITEM REPORT

    Top Menu Items Analysis
Month: all | Category: all

  Rank Menu ID  Menu Item                                Total Sold
------ -------- ---------------------------------------- ----------
     1 C3       Strawberry Creamy Drink                          29
     2 C7       Super-Triple Milk Tea                            26
     3 D3       Ice Cream Toffee Hazelnut Latte                  23
     4 A4       Boba Mi-Shake                                    23
     5 D2       Ice Cream Mocha                                  22
     6 B7       Peach Black Tea                                  22
     7 A1       MIXUE Ice Cream                                  21
     8 A6       Super Mango Sundae                               21
     9 B2       Kiwi Jasmine Tea                                 21
    10 B5       Passion Fruit Bubble Tea                         20
    11 B8       Peach Jasmine Tea                                20
    12 A7       Strawberry Crispy Sundae                         19
    13 A5       Super Boba Sundae                                19
    14 C8       Twin-Topping Milk Tea                            17
    15 A8       O-Crispy Sundae                                  16
    16 C6       O-CoCo Milk Tea                                  16
    17 C2       Brown Sugar Bubble Tea                           15
    18 D1       Ice Cream Latte                                  15
    19 A3       Peach Mi-Shake                                   14
    20 C5       Toffee Hazelnut Milk Tea                         14
    21 C4       Kiwi Creamy Drink                                14
    22 A2       Strawberry Mi-Shake                              14
    23 C1       Classical Milk Tea                               13
    24 B1       Fresh Lemonade                                   13
    25 B4       Lemon Jasmine Tea                                11
    26 B3       Lemon Black Tea                                  11
    27 B6       Creamy Mango Boba                                11
                                                          ----------
TOTAL                                                        480

27 rows selected.
```

6.3.3.2 Result

```
Enter month (1-12, or ALL): 7
Enter category code (or ALL): IC01


     MONTHLY TOP SELLING MENU ITEM REPORT

    Top Menu Items Analysis
Month: 7 | Category: IC01

  Rank Menu ID  Menu Item                                Total Sold
------ -------- ---------------------------------------- ----------
     1 A7       Strawberry Crispy Sundae                          3
     2 A5       Super Boba Sundae                                 2
     3 A6       Super Mango Sundae                                2
     4 A4       Boba Mi-Shake                                     1
     5 A8       O-Crispy Sundae                                   1
     6 A1       MIXUE Ice Cream                                   1
                                                          ----------
TOTAL                                                            10

6 rows selected.
```

Diagram 6.3.3.1 and 6.3.3.1 shows the Monthly Top Selling Menu Item report for a specific month and of a specific category that enter by the users

6.4 Most Ordered Category Report (Soh Jia Le)

6.4.1 Query Purpose

The purpose of this query is to generate a report called "CATEGORY ORDER REPORT" that identifies the most ordered menu categories for a selected month or across all months.

The script prompts the user to input a month, then retrieves data from the category, menu, order_detail, and orders tables to calculate and rank each category based on the total number of orders. If categories have the same order quantity, the ranking is determined by category ID. The results are displayed in a clear table with rankings, column headers, and a summary total at the end.

This report helps management understand customer ordering trends and identify the most popular menu categories, supporting decisions related to inventory planning, marketing focus, and menu management.

6.4.2 Query

```
-- Clear previous settings
CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
TTITLE OFF
BTITLE OFF

-- Set environment
SET ECHO OFF
SET VERIFY OFF
SET FEEDBACK OFF
SET PAGESIZE 40
SET LINESIZE 70

-- INPUT
```

```
PROMPT Filter Options for Most Ordered Category Report:
PROMPT
ACCEPT p_month CHAR PROMPT 'Enter month (1-12, or ALL): '

-- Report title
TTITLE CENTER 'CATEGORY ORDER REPORT' SKIP 1 -
       CENTER 'Most Ordered Categories' SKIP 1 -
       LEFT 'Month: ' '&p_month' SKIP 2

-- Column formatting
COLUMN rank           FORMAT 99       HEADING ' Rank '
COLUMN cat_id         FORMAT A10      HEADING 'Cat ID'
COLUMN description    FORMAT A35      HEADING 'Category'
COLUMN total_orders   FORMAT 999      HEADING 'Orders'

-- Break and compute
BREAK ON REPORT
COMPUTE SUM LABEL 'TOTAL:' OF total_orders ON REPORT  total_orders

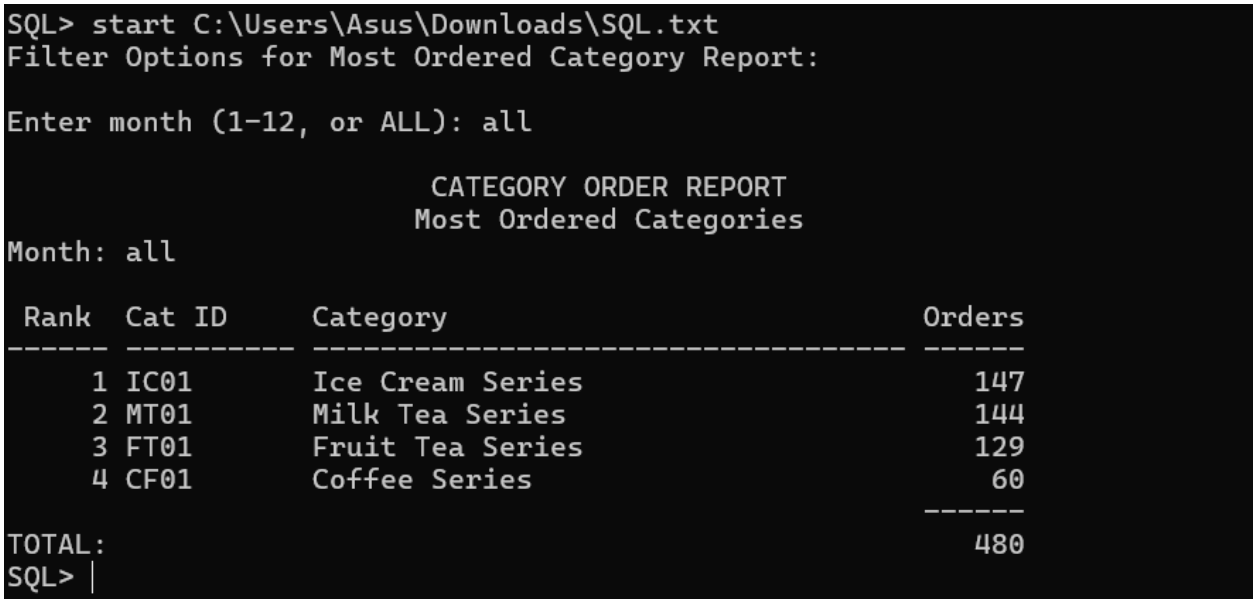
-- Main query
SELECT
   ROW_NUMBER() OVER (ORDER BY COUNT(od.order_no * od.quantity) DESC, c.cat_id)
AS rank,
   c.cat_id,
   c.description,
   COUNT(od.order_no * od.quantity) AS total_orders
FROM
   category c
JOIN menu m ON c.cat_id = m.cat_id
JOIN order_detail od ON m.menu_id = od.menu_id
JOIN orders o ON od.order_no = o.order_no
WHERE
   (UPPER('&p_month') = 'ALL' OR EXTRACT(MONTH FROM o.order_date) =
TO_NUMBER('&p_month'))
GROUP BY
   c.cat_id, c.description
ORDER BY
   rank;

-- Clean up
CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
TTITLE OFF
```

6.4.3 Test Case

Cases	Input/ Test Data	Expected Output	Test Result Status
6.4.3.1	Enter month: ALL	Shows all categories from all months ranked by total order count for the whole year.	Success
6.4.3.2	Enter month: 1	Shows all categories ranked by order count for January only.	Success

6.4.3.1 Result



6.4.3.2 Result

```
SQL> start C:\Users\Asus\Downloads\SQL.txt
Filter Options for Most Ordered Category Report:

Enter month (1-12, or ALL): 1

                        CATEGORY ORDER REPORT
                       Most Ordered Categories
Month: 1

 Rank  Cat ID     Category                          Orders
------ ---------- ---------------------------------- ------
     1 MT01       Milk Tea Series                        15
     2 IC01       Ice Cream Series                       10
     3 FT01       Fruit Tea Series                        8
     4 CF01       Coffee Series                           7
                                                     ------
TOTAL:                                                  40
SQL>
```

Diagram 6.4.3.1 and 6.4.3.2 shows the Most Ordered Category Report for a specific month that enter by the users

6.5 Daily Sales Report (Yuki Soo Zi Qi)

6.5.1 Query Purpose

This query is used to generate a daily sales report on a specific date by displaying order detail, menu, total items sold and total sales amount for that day. It connected data from three tables, which is orders, order_details and menu.

The script first prompts the users to enter a specific day and month (DD / MM). Based on the user inputs, the query will retrieve all the orders that were made on the selected date and calculate the corresponding sales.

This report helps managers to summarise the daily total items and daily total amount for the selected date. As a result, managers can easily identify high or low performing days to support better operation planning and decision-making.

6.5.2 Query

```
SET VERIFY OFF
ALTER SESSION SET NLS_DATE_FORMAT = 'DD/MM/YYYY';
CL SCR

-- User Input --
PROMPT Filter Options for Daily Sales Report:
ACCEPT v_date FORMAT A5 PROMPT 'Enter the date (DD/MM): '

-- Column formatting --
COLUMN order_date    FORMAT A12         HEADING 'Order Date'
COLUMN order_no      FORMAT 9999999     HEADING 'Order No'
COLUMN menu_id       FORMAT A8          HEADING 'Menu ID'
COLUMN description   FORMAT A35         HEADING 'Menu Name'
COLUMN price         FORMAT 999999.99  HEADING 'Price(RM)'
COLUMN quantity      FORMAT 999         HEADING 'Quantity'
COLUMN total_amount  FORMAT 999999.99  HEADING 'Total Amount(RM)'

-- Size --
SET PAGESIZE 50000
SET LINESIZE 120
```

```
-- Report Title --
TTITLE CENTER 'Daily Sales Report for &v_date/2024'

-- Break and Compute --
BREAK ON order_date ON order_no
COMPUTE SUM LABEL 'Daily Total Items'  OF quantity  ON order_date
COMPUTE SUM LABEL 'Daily Total Sales' OF total_amount ON order_date

-- Multi-table Query --
SELECT
   o.order_date,
   o.order_no,
   od.menu_id,
   m.description,
   od.price,
   od.quantity AS quantity,
   od.quantity * od.price AS total_amount
FROM orders o
JOIN order_detail od
   ON o.order_no = od.order_no
JOIN menu m
   ON od.menu_id = m.menu_id
WHERE o.order_date = (TO_DATE('&v_date'||'/2024','DD/MM/YYYY'))
ORDER BY o.order_no
;

-- Clear up --
TTITLE OFF
CLEAR BREAK
CLEAR COMPUTE
```

6.5.3 Test Case

Cases	Input/ Test Data	Expected Output	Test Result Status
6.5.3.1	Enter day/month (DD/MM): 23/3 OR 23/03	Shows total items and total sales amount for one specific day	Success

6.5.3.1 Result

```
SQL> start "C:\CreateTable\CreateTable\daily sales.txt"
```

(i)

```
Filter Options for Daily Sales Report:
Enter the date (DD/MM): 23/3

                                        Daily Sales Report for 23/3/2024
Order Date   Order No Menu ID  Menu Name                          Price(RM) Quantity Total Amount(RM)
------------ -------- -------- ---------------------------------- ---------- -------- ----------------
23/03/2024       1042 B5       Passion Fruit Bubble Tea                 6.00        1             6.00
                      D3       Ice Cream Toffee Hazelnut Latte          5.50        1             5.50
                 1045 B1       Fresh Lemonade                           3.00        1             3.00
                      C2       Brown Sugar Bubble Tea                   5.00        1             5.00
                 1047 A6       Super Mango Sundae                       5.00        1             5.00
                      A8       O-Crispy Sundae                          5.00        1             5.00
************ ********                                                          -------- ----------------
Daily Total                                                                           6            29.50

6 rows selected.

SQL>
```

(ii)

```
Filter Options for Daily Sales Report:
Enter the date (DD/MM): 23/03

                                        Daily Sales Report for 23/03/2024
Order Date   Order No Menu ID  Menu Name                          Price(RM) Quantity Total Amount(RM)
------------ -------- -------- ---------------------------------- ---------- -------- ----------------
23/03/2024       1042 B5       Passion Fruit Bubble Tea                 6.00        1             6.00
                      D3       Ice Cream Toffee Hazelnut Latte          5.50        1             5.50
                 1045 B1       Fresh Lemonade                           3.00        1             3.00
                      C2       Brown Sugar Bubble Tea                   5.00        1             5.00
                 1047 A6       Super Mango Sundae                       5.00        1             5.00
                      A8       O-Crispy Sundae                          5.00        1             5.00
************ ********                                                          -------- ----------------
Daily Total                                                                           6            29.50

6 rows selected.

SQL>
```

Diagram 6.5.3.1(i) and 6.5.3.1(ii) shows the daily sales report for a specific date that enter by the users